

Design of a Digital Odometer Using Hall Effect Sensor

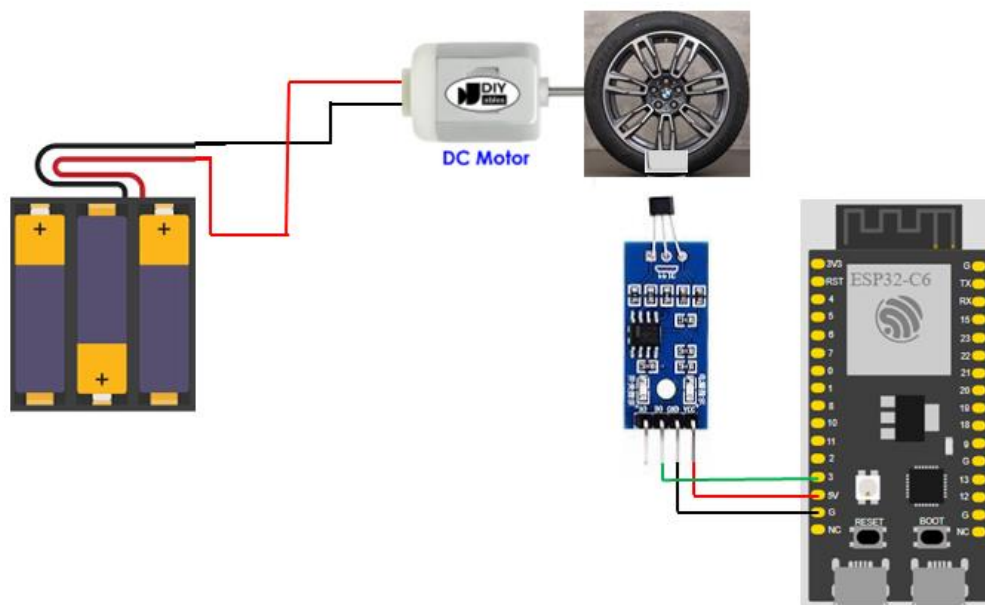
Aim: To design and develop a digital odometer system that measures and displays the distance travelled by a vehicle or rotating wheel using a sensor.

Components required:

Sno	Name of the Component	Quantity
1	ESP32C6 Dev Module	1
2	12V DC Motor	1
3	3.7V 18650 battery	3
4	3x1 18650 battery holder with switch	1
5	Hall sensor	1
6	Magnet	1
7	Jumper Wires	As needed

Description of the project: The Project is designed to measure the total distance travelled by a wheel using a Hall-effect sensor. A magnet is attached to the wheel, and each rotation generates a pulse when it passes the sensor. The microcontroller (ESP32C6) counts these pulses and calculates the total distance based on the wheel's circumference. The calculated distance is then displayed on mobile phone.

Circuit Diagram:



1. Power Supply Connections (Motor Side)

- Connect the positive terminal (+) of the battery pack directly to the positive terminal of the DC motor.
- Connect the negative terminal (–) of the battery pack directly to the negative terminal of the DC motor.
- The motor will start rotating once the battery is connected.

2. DC Motor and Wheel Setup

- Fix the wheel properly onto the motor shaft.
- Ensure the wheel is tightly attached and does not wobble.
- The wheel should have magnet as per the circuit diagram
- This marking or slot is necessary for the sensor to detect rotation.

3. Speed Sensor Module Connections (To ESP32-C6)

The sensor module has 3 pins: VCC, GND, DO

- Connect the VCC pin of the sensor to the 5V pin of the ESP32-C6.
- Connect the GND pin of the sensor to the GND pin of the ESP32-C6.
- Connect the DO (Digital Output) pin of the sensor to a digital GPIO pin 3 of the ESP32-C6.
- The DO pin sends a HIGH and LOW signal whenever the wheel passes in front of the sensor.

4. Sensor Positioning

- Place the sensor in front of the rotating wheel where the magnet is fixed.
- Maintain a small gap of about 3–5 mm between the sensor and the wheel to detect the magnetic field.
- Ensure the sensor faces directly toward the slot or reflective mark.
- When the wheel rotates, the sensor LED may blink indicating detection.

- Proper alignment ensures accurate pulse detection.

5. Signal Working Explanation

- When the wheel rotates, each slot or mark passes in front of the sensor.
- The sensor detects the change and generates a digital pulse.
- The DO pin output changes from LOW to HIGH (or HIGH to LOW).
- The ESP32C6 reads this change using a digital input pin.
- Each pulse represents one rotation.
- By counting pulses for one second, the motor speed (RPM) can be calculated.

6. ESP32C6 Power

1. Power the ESP32-C6 using a USB cable connected to a laptop.
2. Ensure the ESP32C6 operates at 5V logic level, which is compatible with the sensor module.
3. Do NOT power the motor from the ESP32C6 board.

7. Final System Check

- Turn on the motor by connecting the battery.
- Verify that the wheel rotates smoothly.
- Check if the sensor LED blinks when the wheel rotates.
- Open the Serial Monitor in Arduino IDE to verify pulse counting.
- If pulses are not detected:
 - Check wiring connections
 - Verify common ground
 - Adjust sensor distance

Full Code:

```
#include <WiFi.h>
#include <WebServer.h>

#define HALL_PIN 3
#define PULSES_PER_REV 1

// Wheel parameters
#define WHEEL_DIAMETER 0.80 // meters
#define PI_VAL 3.1416
#define WHEEL_CIRCUMFERENCE (PI_VAL * WHEEL_DIAMETER)

const char* ssid = "ODOMeter";
const char* password = "12345678";

WebServer server(80);

volatile unsigned long pulseCount = 0;
unsigned long lastTime = 0;

float rpm = 0;
float distance_m = 0;

// Hall sensor interrupt
void IRAM_ATTR hallISR() {
    pulseCount++;
}

// Web page handler
void handleRoot() {
    String page = "<!DOCTYPE html><html>";
    page += "<head>";
    page += "<meta name='viewport' content='width=device-width, initial-scale=1'>";
    page += "<meta http-equiv='refresh' content='1'>";
    page += "<style>";
    page += "body{font-family:Arial;text-align:center;background:#f2f2f2;}";
    page += "h1{font-size:48px;color:#333;}";
    page += "h2{font-size:28px;color:#555;}";
    page += ".box{background:#fff;padding:20px;margin:20px;border-radius:10px;}";
    page += "</style></head>";
    page += "<body>";

    page += "<div class='box'>";
    page += "<h2>ESP32-C6 RPM & Odometer</h2>";
    page += "<h1>" + String(rpm, 0) + "</h1>";
    page += "<p>RPM</p>";
```

```
page += "</div>";

page += "<div class='box'>";
page += "<h1>" + String(distance_m, 2) + "</h1>";
page += "<p>Distance (meters)</p>";
page += "</div>";

page += "</body></html>";

server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);

  pinMode(HALL_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(HALL_PIN), hallISR, FALLING);

  // Start ESP32-C6 as Access Point
  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  Serial.println("ESP32-C6 AP Started");
  Serial.print("Connect to WiFi: ");
  Serial.println(ssid);
  Serial.print("Open browser: ");
  Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.begin();

  lastTime = millis();
}

void loop() {
  server.handleClient();

  if (millis() - lastTime >= 1000) { // 1 second window
    noInterrupts();
    unsigned long pulses = pulseCount;
    pulseCount = 0;
    interrupts();

    // RPM calculation
    rpm = (pulses * 30.0) / PULSES_PER_REV;

    // Distance calculation
```

```
float rev_per_sec = rpm / 60.0;
float distance_per_sec = rev_per_sec * WHEEL_CIRCUMFERENCE;
distance_m += distance_per_sec;

Serial.print("RPM: ");
Serial.print(rpm);
Serial.print(" | Distance: ");
Serial.print(distance_m, 2);
Serial.println(" m");

lastTime = millis();
}
}
```

Note: The above code is configured for a **Meccano wheel diameter**. If you are using a different wheel, update the wheel diameter value in the code according to the specifications of your wheel.